Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled "Core JDO" are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdobook@yahoogr oups.com

CHAPTER 10

"No one can build his security upon the nobleness of another person"
Willa Cather (1873 - 1947)

Security

Complex applications bring a collection of security requirements to today's data management. These requirements begin at application development time to protect different layers in the software architecture against each other and end at deployment time when production systems have to be protected against malicious access or data corruption. The first part of the chapter defines those different levels of security, their requirements and objectives. A very controversial part of JDO, the so-called Reference Enhancer, will be discussed under security aspects. The rules and measures defined by JDO that keep up the Java sandbox security and secure an application against foreign or malicious code are also examined. The last part of the chapter is about application level security and how J2EE and JDO work together regarding security.

10.1 SECURITY LEVELS

Today's database applications become increasingly complex, which is a reason why JDO has been introduced to make their development simpler. On the other hand this complexity demands a higher security level, because of higher data volumes, more users, more complex data models. When applications provide their services and data over the Internet, a multi-tier system can process or exchange some hundred megabytes per second. Special precautions have to be made to protect sensitive data. By clever planning and partitioning into application layers the protection overhead against reluctant access or data corruption can be reduced. The following paragraphs define the different levels or domains of security and their objectives.

10.1.1. Field and class level security

The Java programming language already provides some security properties, so that code can be written to encapsulate data and deny foreign code access to inner data structures of classes. By using the well-known keywords "private" and "public" field, method and dass access can be extended or limited. Unfortunately there is no difference between "reading" and "modifying", so that programmers often implement `set...()` and `get...()` methods to restrict field access. The C++ const qualifier is also missing.

The objective of field and method access restrictions is primarily to make code more structured, not to protect data access.

An example for this security domain can be found in the collections framework, `java.util.Collection` and friends. Modification or deletion fromcollections should be prevented or at least detected while another part of the application still iterates the collection. Some collection class implementations throw a `ConcurrentModificationException` when this case happens. If the framework developers had allowed access to next and previous references of a list, such detection could not be implemented.

On the other hand the Java VM can provide real security by code verification and different `ClassLoader` instances. This kind, the so-called sandbox security, protects non-public fields and methods against foreign code. Compared to C++, a cast of some reference cannot be used for direct memory access. An exception is the Reflection API, which not only allows field access but also provides meta-data about classes. Since Java 2 the Reflection API is protected by a security policy and has to be enabled for code bases, which need meta-data access. Special considerations have to be taken to disable extension of packages by foreign code, which would withdraw package level security. By "sealing" and signing a JAR file, the simple implementation of another class in the same package can be refused.

10.1.2. Datastore level security

The restriction of database operations on columns, rows, tables or even complete databases is covered by this layer of security. Database systems often implement their own user administration and group, role and rights management. Usually users can be assigned to groups, which have a set of rights. For example, an administrator group may create or delete tables, may make backups and so on, while the employee management may modify salaries or may read other sensitive data.

Basically there are the following operations:

- Read
- Insert
- Modify
- Delete

- Query

These operations can be applied to:

- Columns
- Rows
- Tables
- Users
- Rights/Roles
- Databases

Under certain circumstances the rights management can lead to a particular database design, for instance if there are no column level access rights definable. The test of access rules can have strong performance effects and may also lead to special database designs.

Using cryptography can be necessary either to protect the data against unauthorized copying (fixed disks and backups) or it can protect data that is sent via public networks.

10.1.3. Application level security

Implementing application level security can create a very complex, flexible and powerful access management, which can hardly be realized by Java or database level security. A good design is very important and must be able to prevent inadvertently implemented security holes. In the past years many web site were found, which let intruders execute SQL-code directly from outside. Some developer had overseen a niche or the JDBC user/password pair could be tracked somehow. Often this security hole is introduced by String operations in the code instead of a reasonable encapsulation, for instance when input parameters are put into SQL queries.

Especially JDO provides an outstanding option to implement application-specific security checks by transparent persistence. A user/group/rights management or access control lists can be implemented in short time.

Role-based security is also provided by the J2EE authentication concept and access restrictions can be declared for EJB interfaces on method-level. How JDO and J2EE work together in a secure context will be explained later.

10.2 IMPLEMENTING PERSISTENCE CAPABLE

To make transparent persistence possible, the JDO specification defines the contract between `PersistenceCapable`, the interface that all persistent classes have to implement, and `PersistenceManager`, the central interface of a JDO implementation. But don't worry, how a JDO implementation works inside, is not be explained in this chapter. Because JDO gets direct access privileges for fields and classes, the contract is examined under security aspects. On the one hand the PersistenceManager must be able to read and write fields and gather information about persistence-capable classes, like field types. This is deliberately not implemented using Java Reflection API, but by a more direct interfacing to get highest performance. On the other hand it should not be possible for anybody to read or modify persistence-capable instances. Restrictions like "private" and so on have to be taken into consideration.

The code necessary for persistence-capable classes may be added to the Java source code by tools before a class is compiled or after compilation by modifying Java bytecode. It is a misunderstanding that JDO implementations are required to modify Java bytecode. In essence the JDO expert group has focused high portability and support of various kinds of possible JDO implementations.

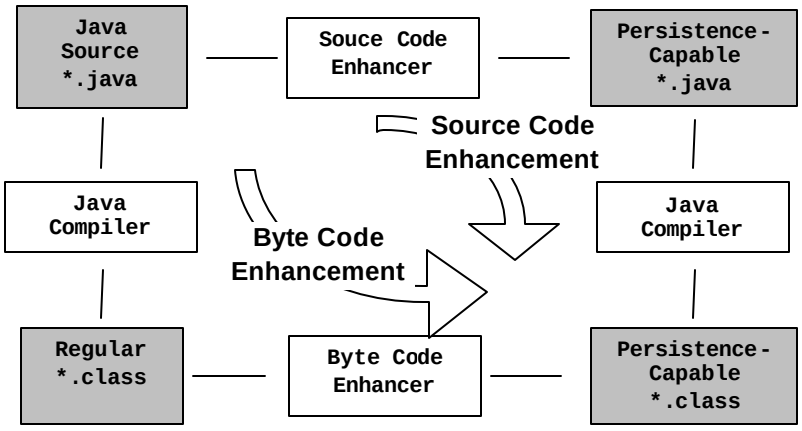

Figure 1: Source Code and Byte Code Enhancement

Using Source Code Enhancement (Figure 1) the Java source files are processed like a C-preprocessor on a text basis. To implement such a source code enhancer a full Java parser is required, because field access and reserved word must be recognized. A simple line-wise

processing is not sufficient. This procedure has the advantage that one can take a look at the resulting source code and debugging is easy, but today's IDEs do not really support source-code pre-processing well enough. It's like early Java Server Page (JSP) development: Without the right hooks in the development environment, edit-compile-debug cycles become a pain.

Bytecode enhancement uses the Java compiler's output and processes `*.class` files. This standardized format, that all Java compilers have to create and that can be interpreted by all Java Virtual machines, is modifiable much simpler that source code. It is even possible to enhance the Bytecode at class load time of the application, like JSPs are compiled at web page load time by the JSP-engine.

A third approach creates persistence-capable classes completely from other sources than Java code. This might be from a database table layout or from XML schema declarations (XSD). The automatically created source code can be persistence-capable inherently.

10.2.1. The Reference Enhancer

The reference enhancer has been an idea to provide an umbrella for different technologies that existed before 1999. Essentially, the API needed to read, modify and create persistence-capable instances could have left vendor-specific, but JDO were not accepted by a broad developer community in that case. During the first phase of the JDO specification, it has been figured out that differences in persistence technologies were insignificant and the reference implementation could also provide a reference enhancer. By specifying the PersistenceCapable interface and the contract between persistence-capable classes and the PersistenceManager the JDO specification achieves the following goals:

10.2.1.1. Binary compatibility

This enables classes to be used by different JDO implementations, once they are enhanced. Persistence-capable applications can change the JDO persistence service without re-compilation or re-enhancement. The JDO compatibility test kit (TCK) takes this into account: Some special tests explicitly check those modifications, which make a class persistence-capable, and compare the Bytecode with reference classes. A JDO implementation can additionally extend the classes, which allows vendors to tune performance for a specific database system or add other vendor-specific features. The reference enhancer is the smallest common denominator.

10.2.1.2. Simplicity

The class must be enhanced by applying minimal code changes. The algorithms defined to make code persistence-capable or persistence-aware, are restricted to exchange only single

opcode in the Bytecode or simply add a method to a class. No complex flow-analysis or structural changes should be needed to make a class persistence-capable.

10.2.1.3. No Reflection Permission

Reflection is needed for JDO implementations only to instantiate an application class. This is done by calling `ClassLoader.getClass(String name)`. All other "meta" information about the classes' fields, key classes etc. are provided by a separate interface that is inaccessible by other than the JDO implementation classes.

10.2.1.4. No Set/Get-methods or persistent base class

The application code is not required to implement any methods to become persistence-capable. The idea is to tag a class as "persistent" and all the rest is done like Java Serialization. A persistence-capable class is also not required to derive from a special, persistent class or interface. The inheritance hierarchy doesn't have to be changed.

10.2.1.5. Field navigation by natural Java syntax

JDO defines names for set and get methods and the rules to convert field access in applications to method invocation. To make a class persistence-aware, field access is changed from

```
String title = book.title;
```

automatically by help from a tool to:

```
String title = book.getTitle();
```

10.2.1.6. Automatic tracking

The application is not responsible anymore to detect modified data and to update the data in a data store or propagate state changes to the JDO implementation. This is done in JDO set()-methods, which are added automatically at enhancement. These methods contain the required code to detect modifications, set flags and propagate state changes.

10.2.1.7. Support of Java field attributes

The application class may use regular field modifiers, like `private`, `public` or even `transient` for any persistent fie lds. In the XML meta-data default attribute for fields may be overridden.

10.2.1.8. High performance

Compared to other persistence technologies, which may use Java Reflection to read or modify persistent instances, JDO provides great performance. As long as an instance is transient or once valid data is loaded from the data store, it performs like any other Java instance. A factory inside of the persistence-capable class is used to create new (hollow) instances without the need of Java Reflection.

10.2.1.9. Same classes for different JDO implementations

The persistence-capable application code can be used together with different PersistenceManager instances at the same time (not the same persistent instances, but the code).

10.2.1.10. Debugger support

A JDO enhancer implementation is required to modify the Bytecode in such a way that any Java Debugger can still be used to debug the application code.

10.2.2. Principles Of Operation

10.2.2.1. Double Dispatch

What has all this to do with security? The main problem is to allow a JDO implementation to access private fields, but protect other code from doing the same. That is the reason for an essential principle in the contract between the persistence-capable class and the PersistenceManager defined in the JDO specification: "double dispatch". Every persistence-capable class has a `StateManager` reference, which is the mediator between itself and the corresponding PersistenceManager. The StateManager is a JDO service provider interface used for dispatching. Principally, the StateManager's methods could have been implemented in the PersistenceManager or in the PersistenceCapable interface. Why this has been designed in such way, will become clear in the next paragraph.

10.2.2.2. Making instances persistent

At the moment a transient instance becomes persistent by a call to `pm.makePersistent(pc)` the JDO implementation firstly has to get the right to access persistent instances to read out fields and store the data persistently. The transactional behavior of the persistent instance has to be controlled by the JDO implementation as well, because the previous state of the instance has to be restored at transaction rollback. At this point the StateManager of the persistent instance is set, and control is granted to the JDO implementation. Just at this point a security check is made through Java's regular security

API, to check the JDO implementation's access permissions. If the JDO's code base is allowed to set the StateManager reference at the persistent instance, it will also get access to all the rest (Figure 2). This ensures only authorized implementations can use the StateManager interface.

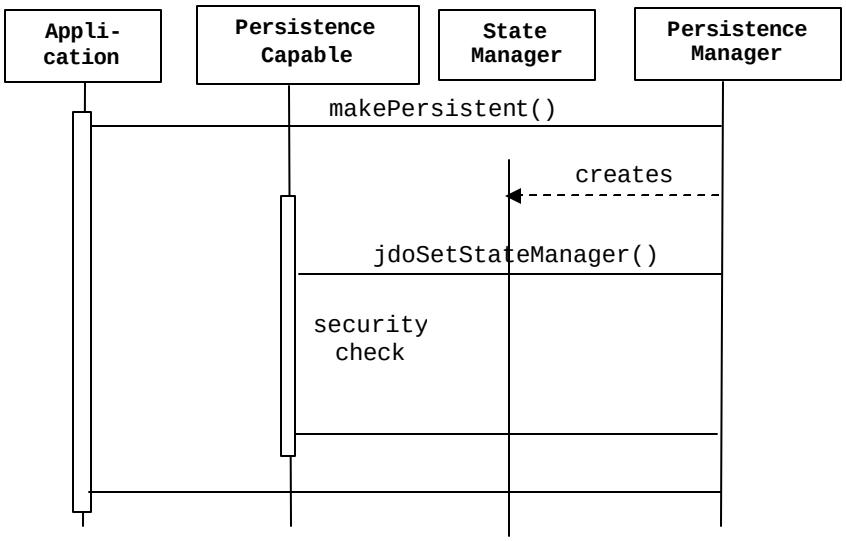

Figure 2: Setting StateManager

The JDO implementation checks initial setting of the StateManager through `JDOPermission("setStateManager")`.

The implementation of the StateManager interface is not defined by JDO. In principle there can be one StateManager instance per persistence-capable instance, one instance for each persistence-capable class or one instance per PersistenceManager instance.

Reading fields from persistent instances

When the instance is made persistent, the PersistenceManager may need to copy fields from the instance into some internal data cache or into the storage. This may happen to save the field data for later restore in case of rollback or at commit time. The PersistenceManager is not allowed to read directly, because any other class could do the same. Instead, it may trigger the operation and field data is provided via the StateManager interface.

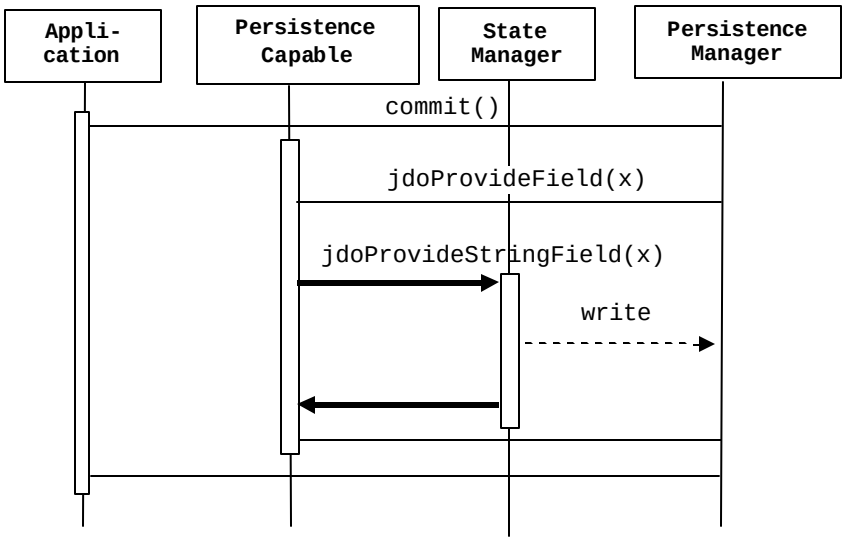

Figure 3: Reading fields

In Figure 3 the PersistenceManager first calls the PersistenceCapable `jdoProvideFields()` method to trigger the read-out by the StateManager, which then receives the data from the persistence-capable instance. The idea is based on the protection of the StateManager: If foreign code cannot set the StateManager reference of the persistence-capable instance, it cannot access fields. The costly Java security check is necessary only once, when a transient instance becomes persistent, or when a hollow instance is created.

The StateManager interface provides all methods to get or provide data from or to the persistent instance. There are some other methods that are called for state changes and modification tracking, which are covered in the next section. The interface might support other applications as well, for instance the Java XML Data Binding (JAXB), which also reads and writes data from/to Java objects (called marshalling and un-marshalling).

10.2.3. Tracking field access

Another important aspect is to track field modifications as well as initial access of hollow instances. Only just resolving hollow instances on demand and mo dification tracking makes database application development with JDO extremely simple. It can be a horror to find a missing line of code in a huge application, which causes some data not being written back into the data store, or to track down performance problems, because data is written too often.

Since many developers are afraid of Bytecode post-processing, the actual code modifications to accomplish persistence awareness are explained here. Firstly, any field access has to be converted into a method invocation. As mentioned earlier, the basic idea is to change a line like

```
String title = book.title;
```

automatically by a tool into

```
String title = book.getTitle();
```

and add a method `getTitle()` that reads the corresponding data from the datastore on demand. The state changes from hollow to persistent-clean in this case. To make persistence-aware classes compatible between different JDO implementations, the name is defined by JDO and the method signature is not the same:

```
String title = Book.jdoGettitle(book);
```

The rule is to add the prefix `jdoGet`, add the field name and declare it as a static method with same access modifiers in the class. The set methods are similar:

```
book.title = "new title";
```

In a persistence-aware class becomes

```
Book.jdoSettitle(book,"new title");
```

To make this a little clearer, the code can be put into a small method, containing just the above lines. The code can be "disassembled" by the `javap` tool after it has been processed by the enhancer. Before enhancement is applied the field assignment disassembles to:

```
0 aload_0
1 getfield #2 <Field Book book>
4 ldc #3 <String "new title">
6 putfield #4 <Field java.lang.String title>
9 return
```

This is the same modification method after the enhancer modified the code looks like:

```
0 aload_0
1 getfield #2 <Field Book book>
4 ldc #3 <String "new title">
6 invokestatic #5 <Method void
                jdoSettitle(Book,java.lang.String)>
```

```
9 return
```

The mutation of the code does not change it's length! Both return statements are at position 9, and debugging information will not change, because mapping of code addresses and source lines is not altered. During access of persistent fields the timing will be changed a bit. So, what happens inside those accessor and mutator (i.e. `get` and `set`) methods? The JDO specification suggests a default implementation, but it is not mandatory. The goal is to run as fast possible in case of transient, persistent-clean and persistent-dirty for fields in the default fetch group.

Default fetch group (DFG)

A fetch group declares a number of fields retrieved from a data store together. Often data is organized in sections, clusters or tables and values can be fetched in a group more efficiently. JDO provides two different types: fields in the default fetch group and fields in other fetch groups. The advantage of the DFG is primarily a flag field that is checked at field access. If that flag is set, the field value is returned directly without calling the JDO implementation. It is not specified how these other groups are declared in the XML meta-data, because it is strongly implementation specific. For example, the Book class might contain a reference to some complex class, that has sub-classes (Category, CategoryList etc). The retrieval of that reference can become expensive in a relational database, because a join query is needed. By declaring the reference in another fetch group, retrieval is delayed until the reference is directly requested by the application.

If a persistent instance needs to be filled with data from a data store, the time necessary to fill the instance is much lengthy than the code in the methods. The proposed `jdoGettitle()` method might be implemented like:

```
final static String
jdoGettitle(Book b) {

  if (book.jdoFlags <= READ_WRITE_OK) {
    return book.title;
  }

  StateManager sm = book.jdoStateManager;

    if (sm != null) {

      // hollow ?
      if (sm.isLoaded(book,jdoInheritedFieldCount+1))
      return book.title;

      return sm.getStringField(book,
              jdoInheritedFieldCount+1,book.title);
```

```
    } else {

      // transient
      return book.title;
    }
  }
```

Firstly, the method checks `jdoFlags` for fields in the default fetch group. If the instance already is filled with valid data from the data store, the instance returns the field. If the StateManager reference was not set yet, the instance is transient, and the field can be returned (else part). If a StateManager is set, the field might be read and returned or the field is completely managed by the JDO implementation and access is always mediated. Here is a summary:

1. The instance is "clean" or "transient"
2. There is no StateManager.
3. The instance is hollow or the field has not been read yet. Access causes the field to be read.
4. Field data is kept in some "shadow" memory and access is always mediated.

The `jdoSettitle()` method works similar and can be implemented in the following way:

```
  final static void
  jdoSettitle(Book b, String value) {

    if (book.jdoFlags == READ_WRITE_OK) {
      book.title = value;
      return;
    }

    StateManager sm = book.jdoStateManager;

    if (sm != null) {
      sm.setStringField(book,
                jdoInheritedFieldCount+1,
                book.title,
                value);
    } else {
      // transient
      book.title = value;
    }
  }
```

Again, the set method checks `jdoFlags` and simply returns if the instance is persistent-dirty already. In this case the title field can be set directly. If a StateManager is set, the appropriate method is called, else the instance is transient and the field can just be set.

The sequence of calls between PersistenceCapable and StateManager allows for flexibility for various kinds of implementations. One extreme can have all fields in the default fetch group. Reading a field causes all other fields to be filled with valid data. Any further read access just returns the corresponding field. The other extreme delegates every access to its JDO implementation. O/R mapping tools probably support a flexible configuration somewhere amid, depending on the mapping of fields to tables. An object-oriented database might always read and write entire objects.

10.2.4. Metadata Access

The `JDOImplHelper` class helps to gather information about persistence-capable classes. It is another service class and is not intended for application use. At the point in time, when a class is initially loaded by the Java VM, it registers itself in `JDOImplHelper`. Later, JDO implementations can query the `JDOImplHelper` singleton for persistent classes and their field attributes. Again, this indirection is needed, because of security issues.

The `JDOImplHelper.getInstance()` method checks for `JDOPermission("getMetadata")` rights and lets only code of JDO implementations execute that have been granted access. If this check was not made, anyone can get information about field names or object identity classes which can be compared to `RuntimePermission("accessDeclaredMembers")`.

10.2.5. Policy File

The JDO Implementation has to be granted privileges to get access to the `PersistenceCapable.setStateManager()` and `JDOImplHelper.getInstance()` methods by adding the following lines to a policy file:

```
grant codeBase "file:/home/myJDOImpl" {
  permission javax.jdo.spi.JDOPermission
        "getMetadata";
  permission javax.jdo.spi.JDOPermission
        "setStateManager";
};
```

10.2.6. Security problems

Unfortunately, the PersistenceCapable interface opens up some security gaps. Since any persistence-capable instance is able to return its PersistenceManager by

```
PersistenceManager pm =
     JDOHelper.getPersistenceManager(book);
```

Any other application part, even some code that has nothing to do with persistence, can operate on the data store. Some code may simply delete the instance from the data store, though it had never been intended. Additionally, other parts of the application can peek at the datastore, because `Transaction`, `Extent`, `Query` and `PersistenceManagerFactory` are easily reached via the `PersistenceManager`.

Sure, this sounds a bit paranoid. On the other hand, JDO leads developers to use persistent objects throughout the application. Compared to direct JDBC programming, which often leads to home-grown persistence layers, persistence-capable instances pass their database connection around. Maybe such a security gap does not really allow malicious code to be executed, but a too exposed API can lead to an application with messed up control paths. To prevent such a mess it is recommended to make instances transient after transaction completion. Under some circumstances it can be an option to pass around JDO object identities instead of persistent instances.

Another security problem can come up from JDOQL: JDO allows querying `private` fields outside of a class or even outside of a package. For example, if the Book class had a private field `purchasePrice`, a simple binary search could find the price for a book. Taking the reference of a persistent `book` instance, the search method can get the `PersistenceManager`, can create Extent and Query instances and run queries. All this has to take place in a running transaction, or a new `PersistenceManager` has to be instantiated from the `PersistenceManagerFactory`.

10.3 APPLICATION SECURITY

The reasons mentioned in the previous section lead to requirements for a clean application design and, if applicable, to copy instance data or to make instances transient before passing them to other application layers. These are the results of the previous section:

- Access modifiers `public`, `private` and `protected` are not changed by the JDO reference enhancer. Fields are still not accessible by other classes, though JDO implementations are authorized to do so by security policies.
- The PersistenceManager is reachable from persistent instances. Those instances can be deleted from or inserted into the data store.
- The other JDO interfaces, `Transaction`, `PersistenceManagerFactory`, `Extent` and `Query` can be reached, and other operations can be performed than those available at the persistent instance.
- Queries can be executed with expressions that contain private fields outside of the owning class.

JDO does provide some basic application security, which can be used to prevent some flaws mentioned above.

10.3.1. User/Password Security

When JDO is used in a non-managed environment, the application can provide the user and password to open a connection to an underlying data store. The code looks like:

```
Properties props = new Properties();
props.setProperty("javax.jdo.option.ConnectionUserName",
                      username);
props.setProperty("javax.jdo.option.ConnectionPassword",
                      password);
... other properties ...
PersistenceManagerFactory pmf =
       JDOHelper.getPersistenceManagerFactory(props);
```

Additionally, the `PersistenceManagerFactory` can create instances of PersistenceManager with different user contexts. As mentioned in chapter ###, it is not possible to change properties of a PMF after a first PersistenceManager has been created.

```
Properties props = new Properties();
... other properties ...
PersistenceManagerFactory pmf =
       JDOHelper.getPersistenceManagerFactory(props);
PersistenceManager pm = pmf.getPersistenceManager(
       username, password);
```

In the second variant, JDO implementations have to keep track of connection pooling and user/manager mapping. If different PersistenceManager instances are used with the same user identity, the same database connection might be used. In the example in Figure 4 the `PersistenceManagerFactory` created three PersistenceManager instances, two of them sharing the same user context.

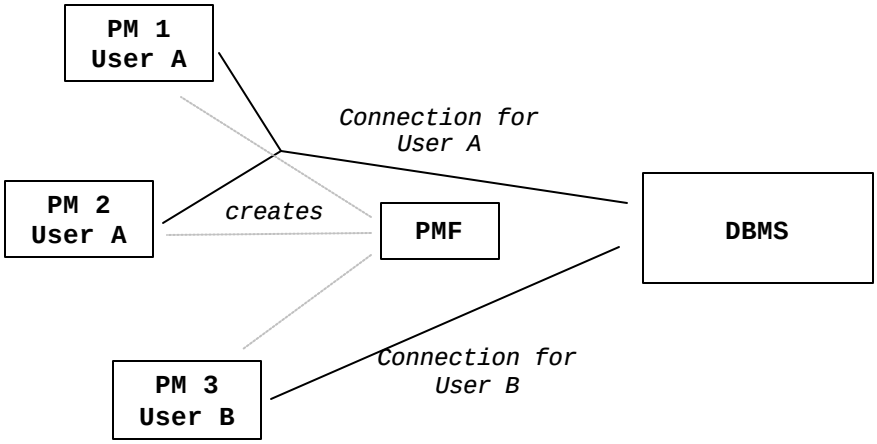

Figure 4: Example: User context and connections

How this mapping is done, is not defined by JDO.

10.3.2. Managed Environments

In managed environments the user or session can be authorized by the container and `PersistenceManagerFactory` instances will be returned by a JCA resource adapter, as explained in the previous chapter:

```
PersistenceManagerFactory pmf =
   (PersistenceManagerFactory)new InitialContext().
        lookup("java:comp/env/jdo/example")
```

The resource adapter is responsible to provide the security context, or gets the security credentials from the container. This means that a session can share its access rights, user

info and so on with different services in the container, like sharing a transaction manager across multiple, distributed transactions. Likewise security checks can be more restrictive in a managed environment, so that only specific classes are allowed to access the persistence service. This would also prevent code to take the PersistenceManager of a persistent instance and create new connections to a data store as mentioned before.

10.3.3. Application specific authorization

Some application types require a different layer of security on top of the data store's native security layer. For example, to manage Internet chat room users by database user administration would be overkill. In this case, it is recommended to implement a simple user administration by means of transparent persistence. Anyhow, whenever rights, groups and users should be managed by the application itself it is advised to do so. It provides data store independency and keeps the domain object model free of vendor-specific code.

10.4 SUMMARY

From the Java language point of view JDO does not introduce any security gaps. The PersistenceCapable interface is well-designed and is optimized for speed as well as requiring effectively the same security checks as Java Reflection does. The JDO programmer is not forced to implement any public set/get methods or to derive from an abstract persistence class. On the other hand, an application developer still needs to keep in mind, that a persistent instance lets anybody access the underlying data store connection, as long as the instance is not made transient. To prevent that, a service-oriented architecture (SOA) is required for security-sensitive applications as explained in the previous chapter.

From the database point of view JDO only provides a very simple username/password authentication. JDO does not define any other access control list (ACL) management. By defining fetch groups it can be possible to read only fields of persistent instances, which the current user is allowed to. This part is completely unspecified by JDO.

Lastly, J2EE managed environments (Servlet, EJB) or application-specific access implementations provide a greater flexibility. They can be used to declare access privileges on a method, class, field or operational basis, but require a higher implementation effort.